

Overview

This release introduces a production-style authentication and authorization framework for the DemoERP API. The system has been upgraded from a hardcoded login approach to a secure, database-backed identity model using JWT authentication, BCrypt password hashing, and role-based + row-level authorization.

This milestone demonstrates enterprise-grade security patterns including authentication, authorization, and multi-user access control.

Authentication & Authorization Model

Data Model

Customer

CRMCustomerID

Users

Username

Role

CustomerAccess

CRMCustomerID | Username

Authorization Flow

Admin?

├── Yes → Allow
└── No

↓



Is (CRMCustomerID, CurrentUser)
present in CustomerAccess?

├── Yes → Allow
└── No → Forbid

🌟 Key Features

Authentication

- Replaced hardcoded credentials with a database-backed `Users` table
 - Implemented user registration endpoint:
 - `POST /api/Auth/register`
 - Implemented secure login endpoint:
 - `POST /api/Auth/login`
 - JWT Bearer token generation with:
 - Username claim
 - Role claim
 - Swagger configured for authenticated API testing
-

Password Security

- Implemented BCrypt password hashing for secure password storage
 - Password verification during login
 - Added hash generation utility endpoint for testing and setup
-

Role-Based Authorization

- Implemented roles:
 - Admin
 - QA
 - Customer
 - Embedded role claims into JWT tokens
 - Applied role-based access control across protected endpoints
-

Row-Level Security (Customer Authorization Model)

Access Rules

Role	Permissions
------	-------------

Admin	Full access to all customer records
QA	Access to QA-owned records and their own
Customer	Access limited to their own records

Security Improvements

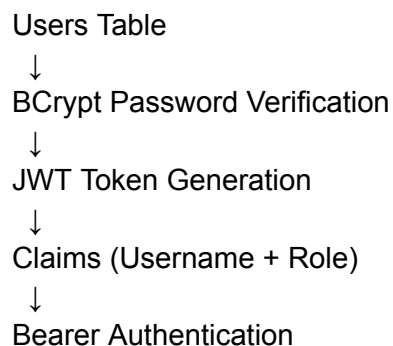
- Removed all hardcoded authentication logic
 - Secured APIs using JWT Bearer authentication
 - Added role claims (Username + Role) to tokens
 - Introduced row-level authorization via **CustomerAccess** table
 - Improved alignment with enterprise API security standards
-

Testing Enhancements

- Added authentication success/failure test cases
 - Added role-based authorization test scenarios:
 - Admin access validation
 - QA restricted access validation
 - Customer self-access validation
 - Verified full CRUD security enforcement
-

System Architecture

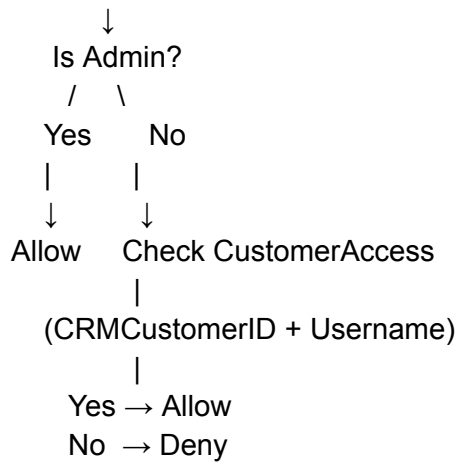
Authentication Flow



↓
Authorization Layer
↓
Customer API
↓
CustomerAccess Validation

Row-Level Authorization Flow

Request → JWT Validation → Role Check



Enterprise Security Features Demonstrated

- JWT Authentication
 - BCrypt Password Hashing
 - Database-backed Identity Management
 - Role-Based Authorization (RBAC)
 - Row-Level Security (Data-level access control)
 - Multi-tenant access design
 - Stored Procedure integration
 - SQL Server integration
 - Soft delete support
 - Audit logging readiness
-

Technical Stack

- ASP.NET Core Web API

- SQL Server
 - ADO.NET (Customer operations)
 - Entity Framework Core (User management)
 - JWT Authentication
 - BCrypt Password Hashing
 - Swagger / OpenAPI
 - Integration Testing
-

Final Result

The DemoERP API now demonstrates a modern enterprise-grade security architecture with:

- Secure authentication system
- Role-based access control
- Row-level authorization
- Database-driven user management
- Protected CRUD operations
- Scalable multi-user access model
- Audit-ready architecture

This milestone significantly improves the security, maintainability, and production readiness of the system.

Interview System Design Explanation Script

1. High-Level Summary

“This system is a secure RESTful API built using ASP.NET Core. It implements JWT-based authentication and supports role-based and row-level authorization for controlling access to customer data.”

2. Authentication Flow

“When a user logs in, their credentials are validated against the Users table. If valid, the system generates a JWT token containing the username and role claims. This token is then used for all subsequent API requests.”

3. Authorization Design

“We implemented two layers of authorization:

First, role-based access control ensures Admin, QA, and Customer users have different permission levels.

Second, row-level security is enforced using a CustomerAccess mapping table, which determines whether a specific user can access a specific customer record.”

4. Data Access Model

“The system uses a many-to-many mapping between users and customers through the CustomerAccess table. This allows multiple users to access the same customer while maintaining strict access control.”

5. Access Decision Flow

“When a request is made, the system first checks if the user is an Admin. If so, access is granted immediately. Otherwise, it checks whether the user has an entry in the CustomerAccess table for that specific customer. If a match exists, access is allowed; otherwise, it is denied.”

6. Security Enhancements

“This design improves security by eliminating hardcoded credentials, introducing BCrypt password hashing, and enforcing token-based authentication. It also ensures that all data access is controlled at both the role and data row level.”

7. Closing Statement

“This architecture reflects a production-grade approach to API security by combining authentication, authorization, and row-level access control in a scalable and maintainable way.”